Projeto Prático 2 – Redução de Dimensionalidade

Representações compactas para classificar e pesquisar peças de roupa parecidas

André Fazendeiro – n.º 48191
Machine Learning, Universidade da Beira Interior, 2026/27

Resumo

Comprimimos as imagens do Fashion-MNIST (784 píxeis) em representações de 2 a 128 números com PCA e com um autoencoder, e usámo-las em duas tarefas: classificar a peça e pesquisar peças parecidas (*look-alike search*). O código do autoencoder é a melhor representação em quase tudo: com 128 números classifica melhor do que os píxeis brutos (85,8% contra 83,3%) e com só 8 já pesquisa tão bem como eles. Mas classificar e pesquisar não são a mesma coisa: o HOG é o 2.º melhor espaço a classificar e o 10.º a pesquisar, e a pesquisa estabiliza com 32 dimensões, enquanto a classificação precisa de 64 a 128. Num subconjunto de fotos a cores, cada descritor ganha num tipo de semelhança diferente (cor, tipo de peça ou um compromisso), o que mostra que o "melhor espaço" depende do que se entende por "parecido".

1 Introdução

A pergunta do projeto é: *que representações compactas das imagens preservam a informação de classe e permitem uma pesquisa útil de peças parecidas?* Para lhe responder, construímos várias representações (PCA, autoencoder, HOG e, para a cor, histogramas) e avaliámo-las em duas tarefas:

- **Classificação:** uma regressão logística, igual para todas as representações, tenta adivinhar a classe.
- **Pesquisa (retrieval):** dada uma imagem, devolver as k peças mais próximas de uma galeria, sem usar labels.

A ideia que queríamos testar é que **a melhor representação para classificar não é necessariamente a melhor para pesquisar**. O código está no repositório https://github.com/Andrepf03/PROJETO2 e corre no Google Colab.

2 Dados e preparação

Fashion-MNIST. 70 000 imagens 28×28 em cinzento, 10 classes, perfeitamente equilibradas (6000 por classe no treino e 1000 no teste). Dividimos os píxeis por 255, ficando em $[0, 1]$. Por ser uma constante fixa, isto não causa *data leakage*. Cada imagem passa a ser um vetor de 784 valores. Dos 60 000 de treino tirámos 10 000 para validação (estratificado), ficando com **50 000 treino / 10 000 validação / 10 000 teste**. A validação serviu para as decisões (por exemplo, o número de épocas). O teste só foi usado no fim, para os números que reportamos. Todas as seeds estão fixas (`SEED = 42`), por isso os resultados são reprodutíveis.

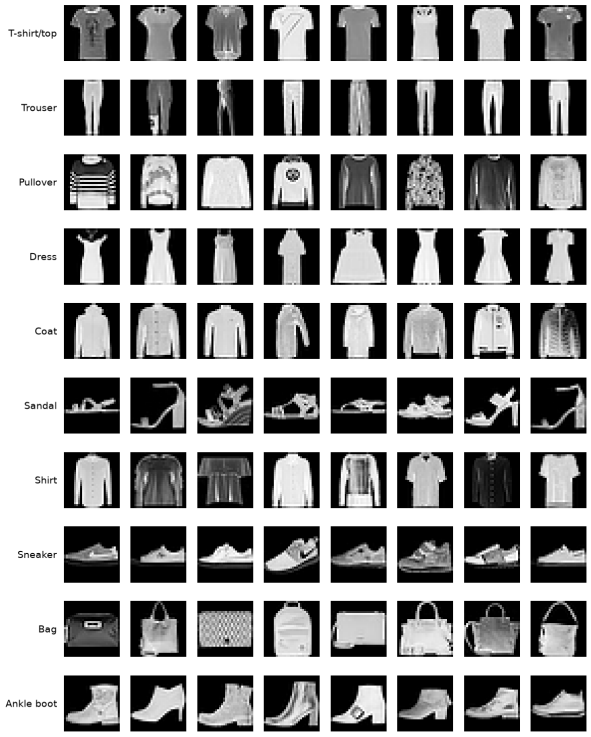


Figura 1: Exemplos de cada classe. As peças de tronco (T-shirt, Pullover, Coat, Shirt) já se notam muito parecidas.

Subconjunto a cores. Para a perspetiva de cor usámos o *Fashion Product Images (Small)*, através da cópia no Hugging Face (`ashraq/fashion-product-images-small`, original no Kaggle). Escolhemos 5 tipos de peça (Tshirts, Shirts, Tops, Casual Shoes, Handbags) e 6 cores (Black, White, Blue, Red, Green, Yellow), com no máximo 100 imagens por combinação, o que deu **2704 imagens**. Cada foto foi posta num quadrado branco (para não deformar) e reduzida para 28×28 RGB. Dividimos 80/20 em galeria (2163) e queries (541), estratificado por tipo e cor. Há três limitações a ter em conta: a roupa de tronco é fotografada vestida por modelos (aparecem cara, pele e calças), as peças brancas confundem-se com o fundo e o amarelo é raro.

3 Métodos

- **PCA:** ajustado só no treino, com todas as componentes. Para d dimensões usamos as primeiras d.
- **t-SNE e UMAP:** só para visualizar (5000 imagens de treino, 500 por classe). Não servem para classificar nem pesquisar, porque não projetam imagens novas e as distâncias entre grupos não têm significado.
- **Autoencoder:** MLP 784–512–256–d–256–512–784, com ReLU nas camadas escondidas e Sigmoid na saída. Treinado com MSE, Adam (lr 10^{-3}), batches de 256 e 50 épocas, guardando os pesos da melhor época na validação. Treinámos um modelo para cada $d \in \{2, 8, 16, 32, 64, 128\}$.
- **HOG (textura/contornos):** 9 orientações, células de 7×7 píxeis e blocos de 2×2, o que dá 324 valores. Não tem treino.
- **Histograma de cor:** RGB com 4 níveis por canal (64 caixas), ignorando os píxeis quase brancos (fundo) e normalizado para somar 1.

- **Classificador:** `StandardScaler` + regressão logística, igual em todos os espaços, para a comparação ser justa.
- **Pesquisa:** galeria = treino, queries = teste, k vizinhos mais próximos com distância euclidiana (força bruta). **As labels nunca entram no ranking**, só na avaliação.
- **Precision@k:** fração dos k primeiros resultados com a mesma classe da query, em média sobre todas as queries. Ao acaso daria 0,10.

O código está organizado em `src/representation.py` (PCA, autoencoder e HOG), `src/retrieval.py` (pesquisa e precision@k), `src/color_data.py` (dataset a cores) e `src/utils.py`, e é corrido em notebooks numerados (00 a 16).

4 Resultados

4.1 PCA: variância explicada

As duas primeiras componentes já explicam 46,8% da variância (29,0% + 17,8%). Para 80/90/95/99% são precisas 24/84/187/459 componentes.

Dimensão	2	8	16	32	64	128
Variância acumulada	46,8%	69,3%	76,5%	82,6%	88,1%	92,8%

Tabela 1: Variância explicada pelo PCA nas dimensões do enunciado.

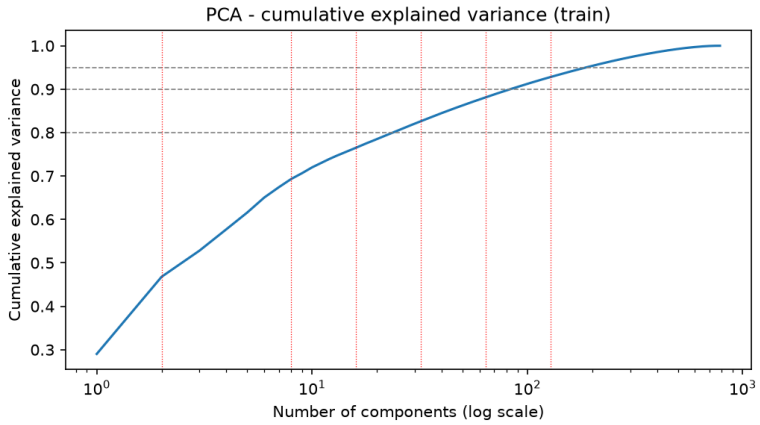


Figura 2: Variância acumulada (eixo x em escala log). A curva sobe depressa até 30 componentes e depois cresce devagar.

Em resumo: a maior parte da informação está em poucas direções. Com 128 componentes (16% das dimensões) guardamos quase 93% da variância.

4.2 Mapas 2D

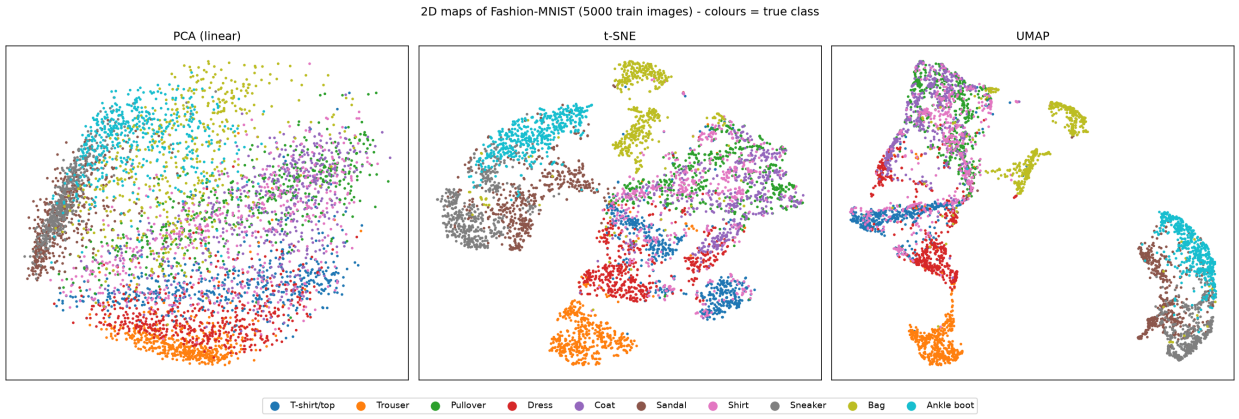


Figura 3: 5000 imagens de treino em 2D com PCA, t-SNE e UMAP. As cores são as classes verdadeiras, que os métodos não usam.

O PCA em 2D só separa as famílias grandes (calçado, calças, malas, roupa de tronco). O t-SNE e o UMAP separam muito melhor, mas mesmo eles misturam Shirt, Pullover e Coat. Ou seja, o problema não é só o PCA ser linear: os próprios píxeis têm pouca informação para distinguir estas peças. A classe Bag aparece dividida em dois grupos, o que volta a aparecer nos casos de falha.

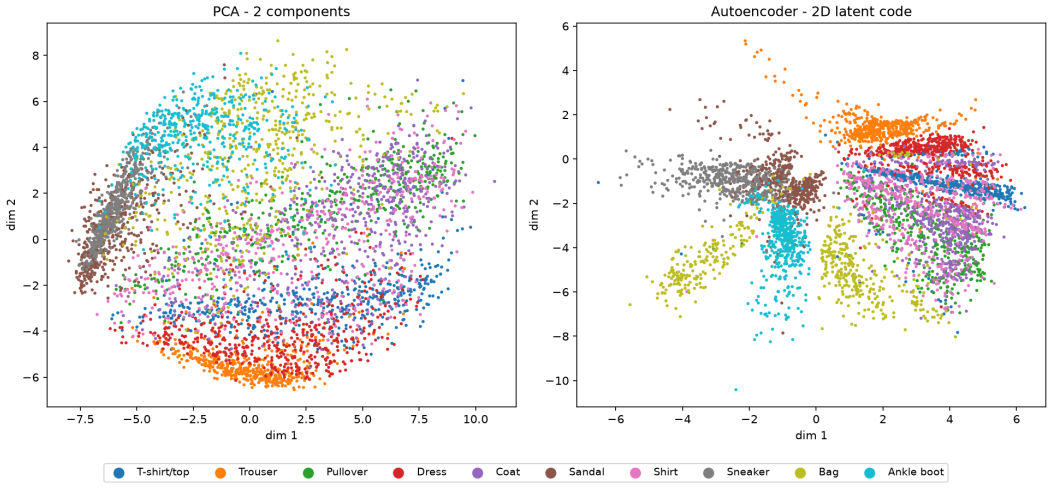


Figura 4: PCA com 2 componentes vs código 2D do autoencoder (5000 imagens de validação).

Com só 2 números, o autoencoder já organiza as peças em "braços" (calçado, calças, vestidos, malas), enquanto o PCA em 2D mistura quase tudo.

4.3 Reconstrução

Dimensão	2	8	16	32	64	128
PCA	0,0461	0,0266	0,0205	0,0152	0,0104	**0,0063**
Autoencoder	**0,0240**	**0,0125**	**0,0100**	**0,0080**	**0,0074**	0,0070

Tabela 2: Erro de reconstrução (MSE por píxel) no conjunto de teste.

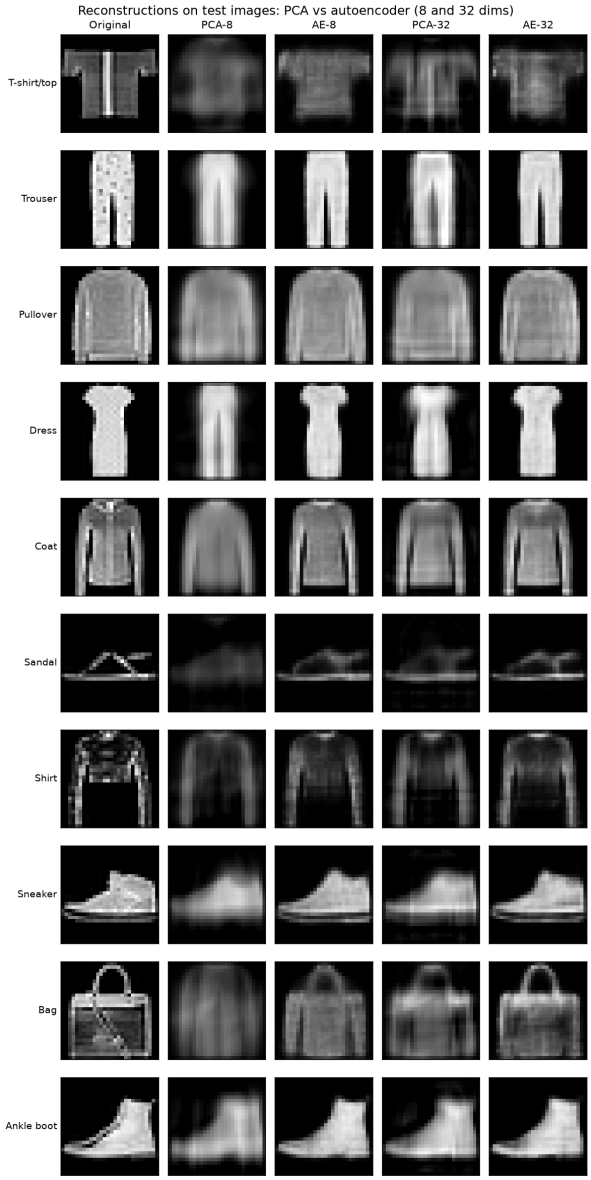


Figura 5: Original vs reconstruções PCA e autoencoder com 8 e 32 dimensões (uma imagem de teste por classe, escolhida ao acaso).

O autoencoder reconstrói muito melhor com poucas dimensões: com 8 números faz o que o PCA faz com cerca de 50. Com 128 ficam praticamente empatados, porque o PCA já guarda quase toda a variância. Nas imagens, o PCA-8 é um "fantasma" borrado, enquanto o AE-8 já mostra a silhueta, a alça da mala ou as tiras da sandália. Mas **os dois perdem padrões e texturas** (riscas, xadrez, estampados): guardam sobretudo a forma. Bag e Sandal são as classes mais difíceis de reconstruir e Trouser a mais fácil. Os erros no teste são quase iguais aos da validação, por isso os modelos generalizam bem.

4.4 Classificação

Dimensão	2	8	16	32	64	128
PCA	0,509	0,730	0,787	0,815	0,832	0,841
Autoencoder	**0,617**	**0,766**	**0,801**	**0,823**	**0,843**	**0,858**
Píxeis brutos (784): 0,833			HOG (324): 0,844			

Tabela 3: Accuracy no teste (regressão logística igual para todos os espaços).

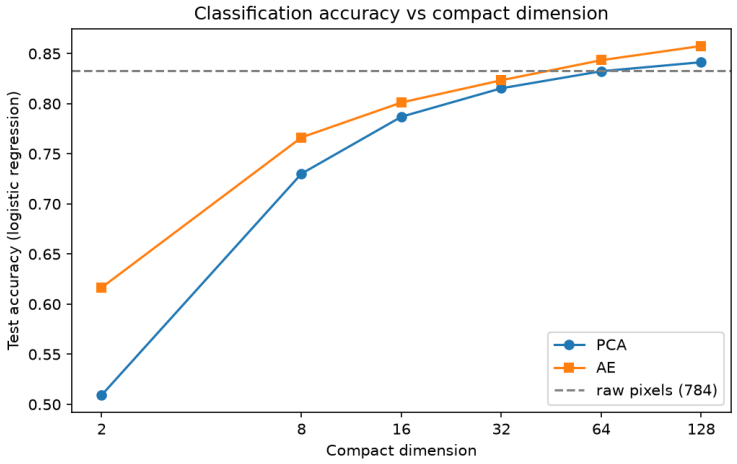


Figura 6: Accuracy vs dimensão. A linha tracejada são os píxeis brutos.

O autoencoder ganha ao PCA em todas as dimensões, sobretudo nas pequenas (+11 pontos com $d = 2$). Com 64 dimensões, os dois já igualam os píxeis brutos, e o AE-128 ultrapassa-os (0,858 contra 0,833). Os píxeis brutos caem mais da validação para o teste (0,856 → 0,833) do que os espaços compactos: com 784 features, a regressão logística decora pormenores do treino. Comprimir funciona como uma limpeza de ruído. O treino nos píxeis demorou 820 s, contra 1–5 s nos espaços compactos. As classes mais difíceis são Shirt, Pullover e Coat (ver a matriz de confusão no Apêndice B).

4.5 Pesquisa de peças parecidas

Dimensão	2	8	16	32	64	128
PCA	0,451	0,728	0,778	0,797	0,803	0,806
Autoencoder	**0,671**	**0,801**	**0,812**	**0,818**	**0,821**	**0,824**
Píxeis brutos (784): 0,800			HOG (324): 0,782		Acaso: 0,10	

Tabela 4: Precision@10 no teste (galeria = treino). A precision@5 segue a mesma ordem e é 1 ponto mais alta.

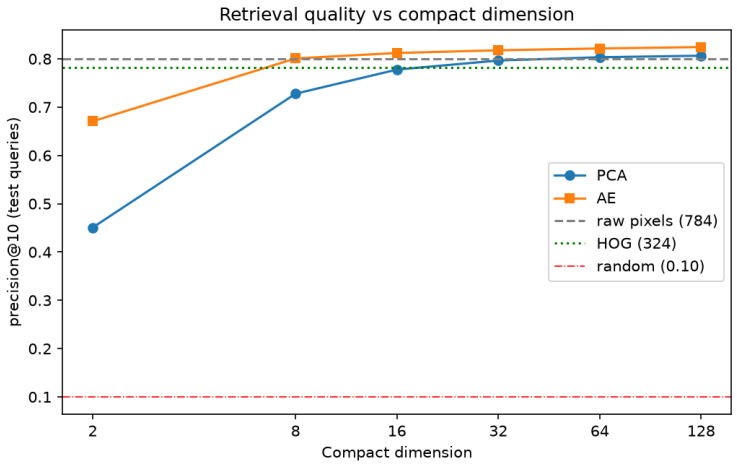


Figura 7: Precision@10 vs dimensão.

O autoencoder é outra vez o melhor espaço, e **com só 8 números pesquisa tão bem como os 784 píxeis**. Na pesquisa, o AE estabiliza por volta de 16–32 dimensões. O HOG, que era o 2.º melhor a classificar, fica abaixo dos píxeis brutos.

Classe	Forma (PCA-32)	Contornos (HOG)	Aprendido (AE-32)
T-shirt/top	0,738	0,737	**0,762**
Trouser	0,951	0,944	**0,956**
Pullover	0,660	0,625	**0,698**
Dress	0,809	0,713	**0,815**
Coat	0,669	0,663	**0,688**
Sandal	0,839	0,818	**0,875**
Shirt	0,543	0,537	**0,570**
Sneaker	0,888	0,909	**0,923**
Bag	0,944	0,934	**0,949**
Ankle boot	0,925	0,940	**0,943**

Tabela 5: Precision@10 por classe nos três espaços de pesquisa.

Em média, o AE-32 é o melhor em todas as classes. Mas os painéis mostram que **cada espaço vê a semelhança à sua maneira**. O PCA e o autoencoder privilegiam o tom de cinzento e a silhueta. O HOG privilegia o contorno e o padrão, e ignora o brilho. Por isso, numa camisa estampada só o HOG encontra outras camisas estampadas (3/5 contra 1/5), e num pullover liso o HOG devolve cinco casacos brancos com o mesmo contorno (0/5).

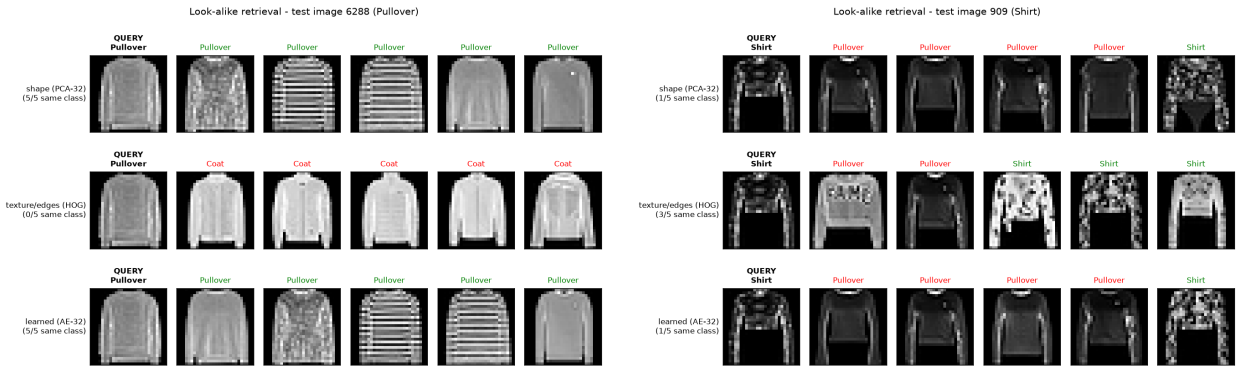


Figura 8: A mesma query pesquisada por forma (PCA-32), contornos (HOG) e embedding aprendido (AE-32). Verde = mesma classe, vermelho = classe diferente. Os restantes painéis estão no Apêndice A.

4.6 Extensão a cores

No dataset a cores comparámos separadamente três espaços: **cor** (histograma RGB), **textura/contornos** (HOG na versão em cinzento) e um **embedding aprendido** (autoencoder com a mesma arquitetura, entrada de 2352 valores e código de 32, treinado só com a galeria). Medimos duas precisões: se os resultados têm a *mesma cor* e se são o *mesmo tipo de peça*.

Espaço	Mesma cor (p@10)	Mesmo tipo (p@10)
Acaso	0,172	0,201
Histograma de cor	**0,680**	0,435
HOG (cinzento)	0,255	**0,751**
Autoencoder a cores	0,506	0,750

Tabela 6: Pesquisa no dataset a cores: cada espaço ganha numa semelhança diferente.



Figura 9: Mala vermelha: o histograma devolve coisas vermelhas (incluindo sapatos), o HOG devolve malas de todas as cores com a mesma forma e o autoencoder devolve malas vermelhas. Verde = mesmo tipo e cor, laranja = só um dos dois, vermelho = nenhum.

Cada descritor ganha na semelhança para que foi feito. O histograma encontra a mesma cor, mas não o mesmo tipo de peça. O HOG encontra o mesmo tipo, mas é cego à cor. O autoencoder iguala o HOG no tipo e vê cor razoavelmente, por isso é o melhor compromisso. Nas fotos com modelos, a pose e a pessoa (cabelo, braços, calças) também influenciam a semelhança, o que é uma limitação de usar fotos sem recortar a peça.

4.7 Casos de falha

No espaço AE-32, 14,8% das queries de teste têm o vizinho mais próximo de outra classe. Quase todas estas confusões são entre peças de tronco, e são simétricas:

Classe da query	Classe do 1.º vizinho	N.º de casos
T-shirt/top	Shirt	158
Shirt	T-shirt/top	145
Coat	Pullover	135
Pullover	Coat	111
Shirt	Pullover	104
Pullover	Shirt	104

Tabela 7: Pares de classes mais confundidos na pesquisa (AE-32, 1.º vizinho).

Escolhemos os casos de forma sistemática, e não "à mão".

Tipo 1 – parecidas, com labels diferentes: para cada par de classes, a query mais parecida com um vizinho de outra classe (6 casos). Estão na Figura 10: dois hoodies quase iguais (um "Pullover", outro "Coat"), polos com gola ("T-shirt" vs "Shirt"), peças de manga comprida com padrão aos quadrados ("Coat" vs "Shirt") e peças brancas lisas ("Pullover" vs "Shirt"). Muitas são ambíguas até para uma pessoa.

Tipo 2 – mesma label, estilos diferentes: k-means com 2 grupos dentro de uma classe (3 casos, Figura 11 e Apêndice B). Bag separa-se em clutches sem alça (2616) e malas com alça alta (2384). Sandal separa-se em sandálias rasas (2760) e de salto alto (2240). T-shirt separa-se em escuras ou estampadas (2330) e claras ou lisas (2670), ou seja, pelo brilho e não pela forma.

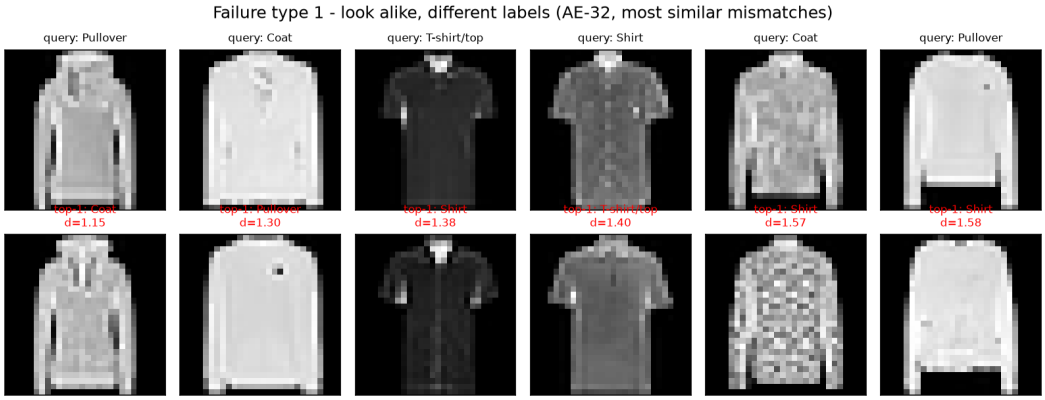


Figura 10: Tipo 1: query (em cima) e o seu vizinho mais próximo de outra classe (em baixo).

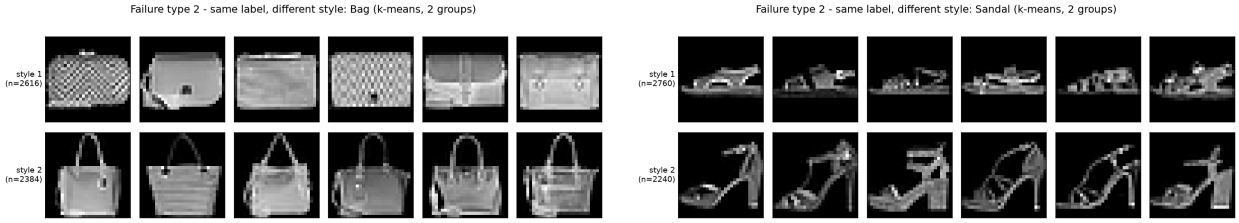


Figura 11: Tipo 2: dois estilos dentro da mesma classe (Bag e Sandal).

O que isto diz sobre o Fashion-MNIST: os detalhes que separam as peças de tronco (botões, fechos, golas, material, cor) não cabem em 28×28 a cinzento. Algumas classes juntam estilos muito diferentes, e nos espaços baseados em píxeis o brilho pesa demasiado. A cor resolveria só uma parte: ajuda a separar peças de cores diferentes, mas não um casaco cinzento de um pullover cinzento.

5 Discussão: classificar ≠ pesquisar

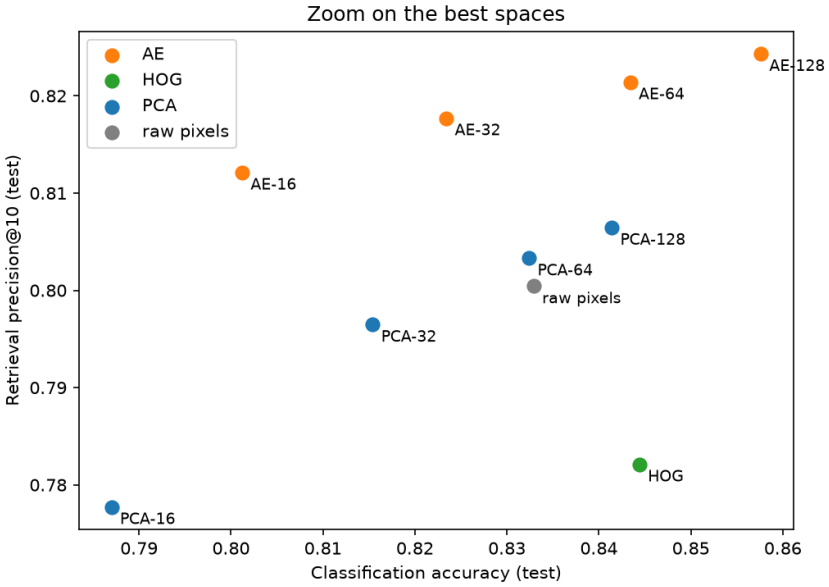


Figura 12: Cada ponto é um espaço: accuracy (x) vs precision@10 (y). Os autoencoders ficam sempre acima dos PCA, ou seja, com a mesma accuracy pesquisam melhor. O HOG é o caso extremo: bom a classificar, fraco a pesquisar.

	Melhor accuracy	Satura em	Melhor p@10	Satura em
PCA	0,841	64	0,806	**32**
Autoencoder	0,858	128	0,824	**32**

Tabela 8: Dimensão a partir da qual cada tarefa fica a menos de 1 ponto percentual do seu melhor valor.

- **A ordem dos espaços muda.** A correlação de Spearman entre a ordem na classificação e na pesquisa é 0,71 (0,84 sem o HOG). O HOG passa de 2.º para 10.º. Os AE pequenos fazem o contrário: o AE-16 é 9.º a classificar e 4.º a pesquisar. O AE-16 classifica pior do que o PCA-32 (0,801 contra 0,815), mas pesquisa melhor (0,812 contra 0,797).
- **Porquê?** O classificador aprende a pesar as features e ignora as que não interessam. O k-NN usa a distância bruta, em que todas as dimensões contam por igual. Um espaço pode ter a informação de classe "lá dentro" sem que as distâncias a reflitam bem.
- **A pesquisa precisa de menos dimensões** (32) do que a classificação (64–128). Para o k-NN basta que peças parecidas fiquem perto, e isso já acontece com poucas dimensões bem organizadas.
- **Na cor, o "melhor" depende da pergunta:** "outra mala vermelha" leva ao histograma, "outra mala com esta forma" leva ao HOG ou ao autoencoder.
- **Sendo honestos:** no Fashion-MNIST, o AE-128 é o n.º 1 nas duas tarefas. A conclusão não é "o melhor para classificar perde sempre na pesquisa", mas sim que *não é necessariamente o mesmo*: a ordem, o número de dimensões necessário e o tipo de semelhança mudam.

6 Conclusão e limitações

Comprimir as imagens funciona: com 32 a 128 números classificamos e pesquisamos tão bem ou melhor do que com os 784 píxeis. O código do autoencoder é a melhor representação em quase tudo, sobretudo com poucas dimensões. A melhor representação para classificar não é necessariamente a melhor para pesquisar: a ordem dos espaços muda, a pesquisa satura mais cedo e, com cor, cada descritor responde a uma ideia diferente de "parecido".

Limitações: medimos a semelhança com a classe, o que é só uma aproximação (há peças quase iguais com labels diferentes e peças muito diferentes com a mesma label). O autoencoder é um MLP simples. O dataset a cores é pequeno, tem fotos com modelos e algum ruído nas labels de cor.

Trabalho futuro: um autoencoder convolucional (deve reconstruir melhor os detalhes), embeddings de uma CNN pré-treinada, histogramas em HSV e recortar a peça antes de calcular a cor.

A Apêndice A – Painéis de retrieval

Uma query de teste por classe, escolhida ao acaso com seed fixa (não foram escolhidas "à mão"). Verde = mesma classe da query, vermelho = classe diferente.

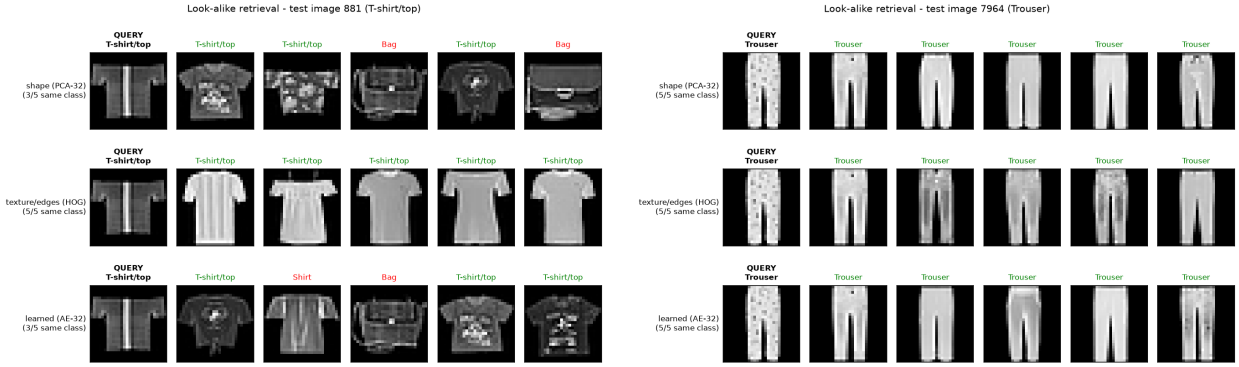


Figura 13: T-shirt/top e Trouser.

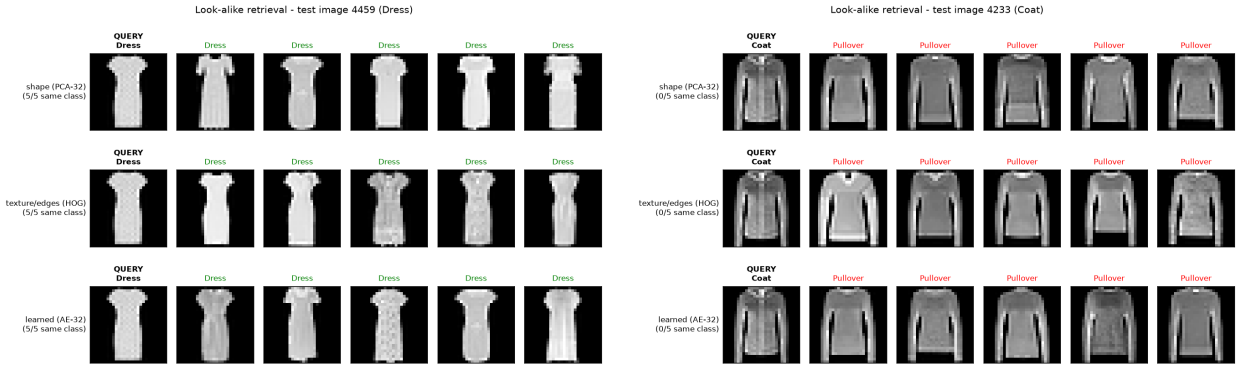


Figura 14: Dress e Coat (o Coat é confundido com Pullover nos três espaços).

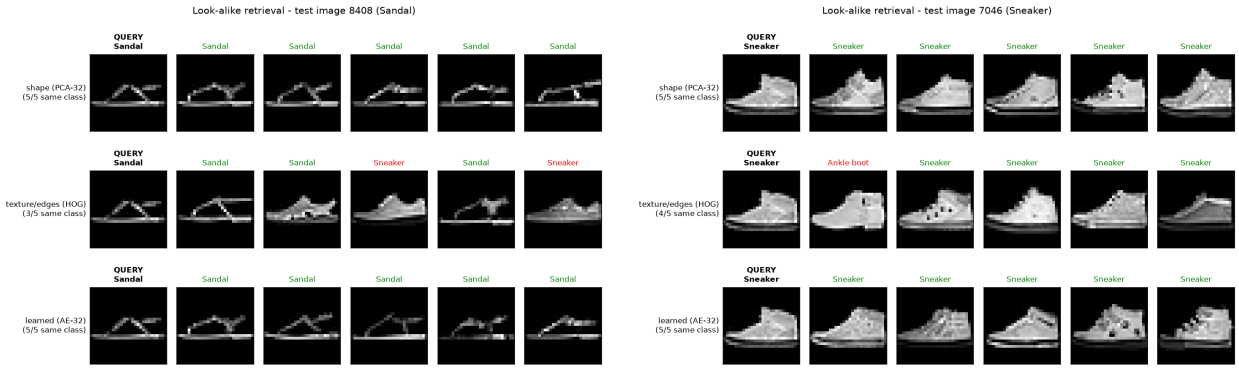


Figura 15: Sandal e Sneaker.

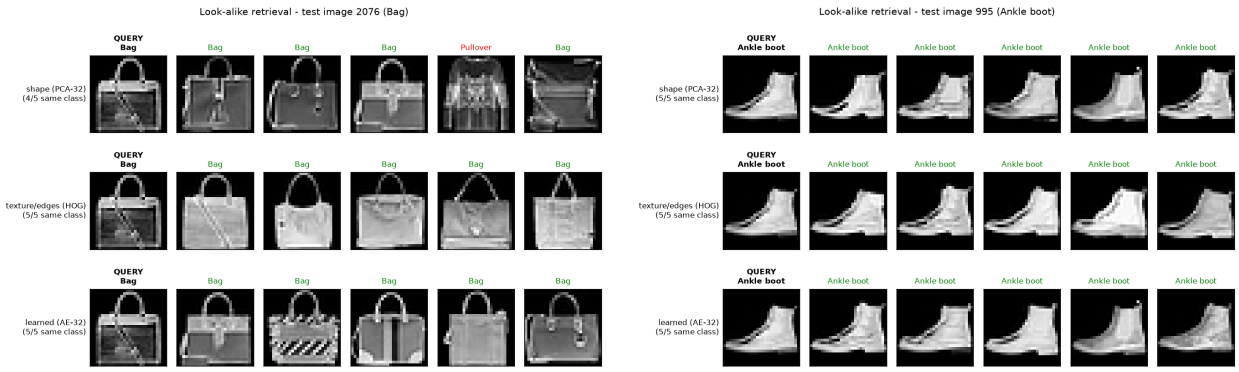


Figura 16: Bag e Ankle boot.

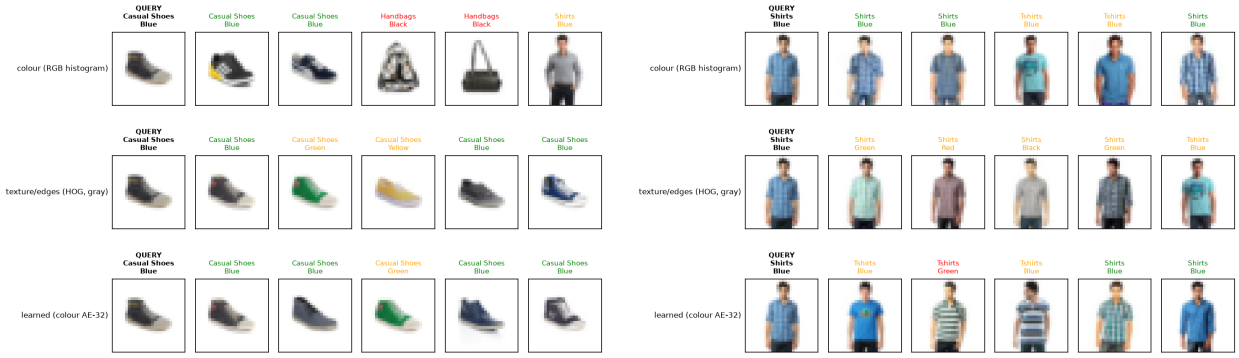


Figura 17: Dataset a cores: Casual Shoes e Shirts.

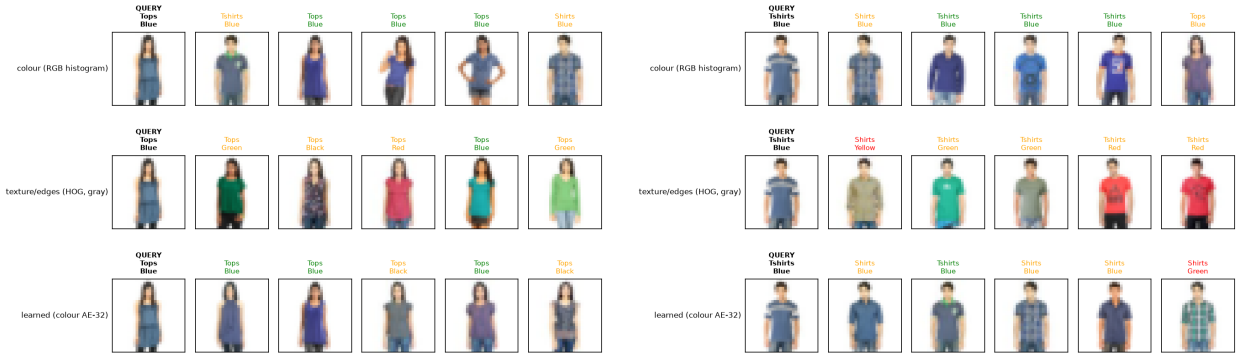


Figura 18: Dataset a cores: Tops e Tshirts. O HOG e o autoencoder apanham a pose e o cabelo da modelo.

B Apêndice B – Figuras complementares



Figura 19: Subconjunto a cores: um exemplo por tipo e cor.

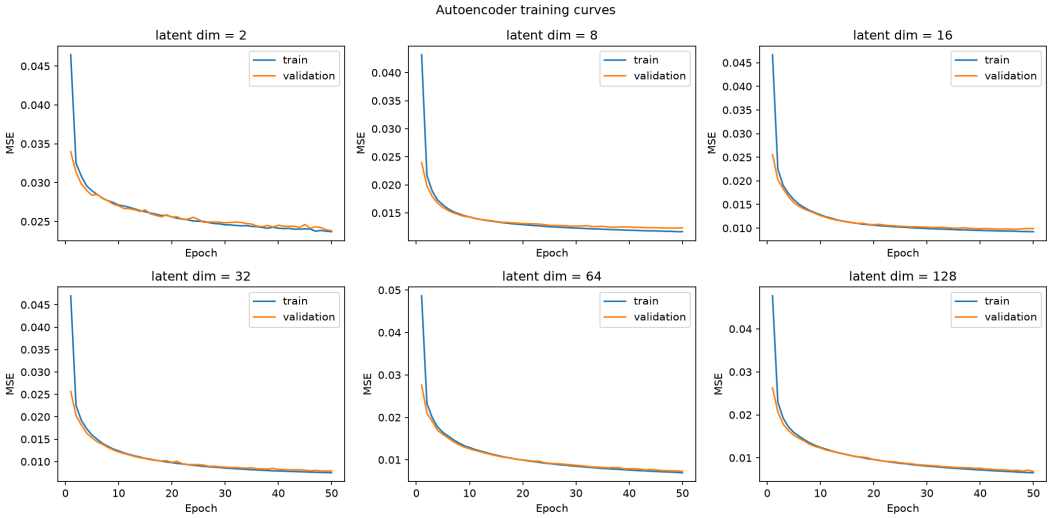


Figura 20: Curvas de treino dos autoencoders (50 épocas). Não há overfitting relevante. A melhor época ficou entre 46 e 50.

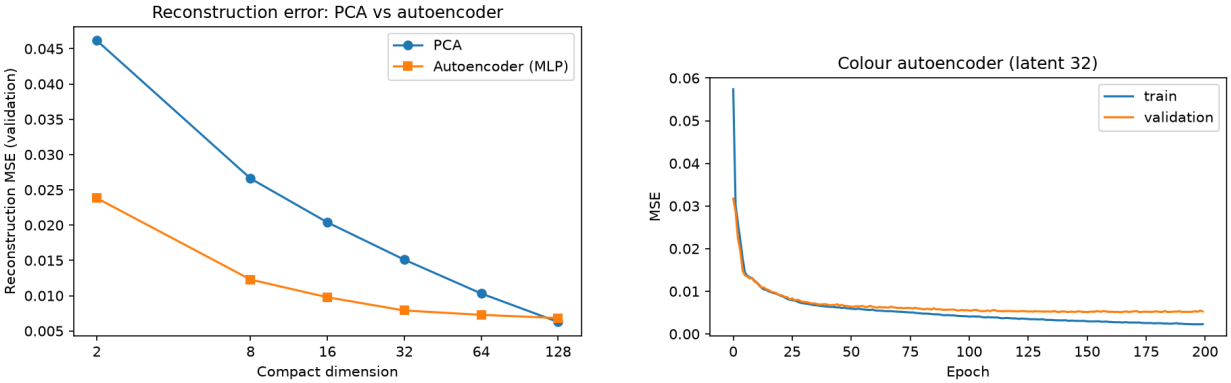


Figura 21: Esquerda: erro de reconstrução PCA vs autoencoder (validação). Direita: treino do autoencoder a cores (melhor época: 165).

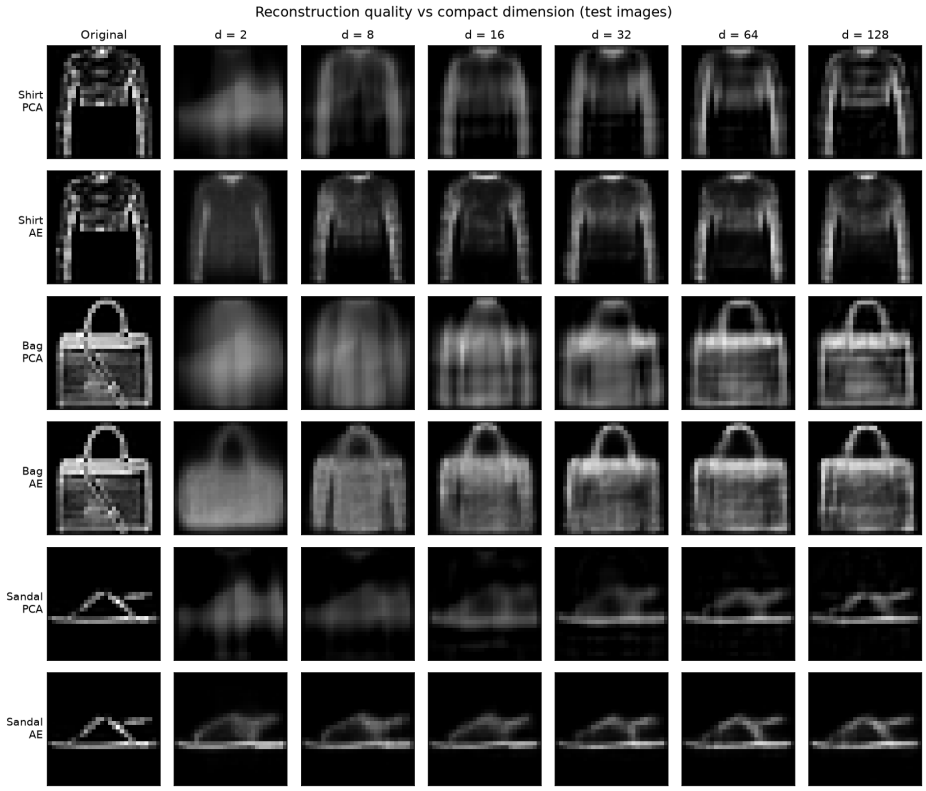


Figura 22: Reconstrução de três peças com detalhe (Shirt, Bag, Sandal) de 2 a 128 dimensões.

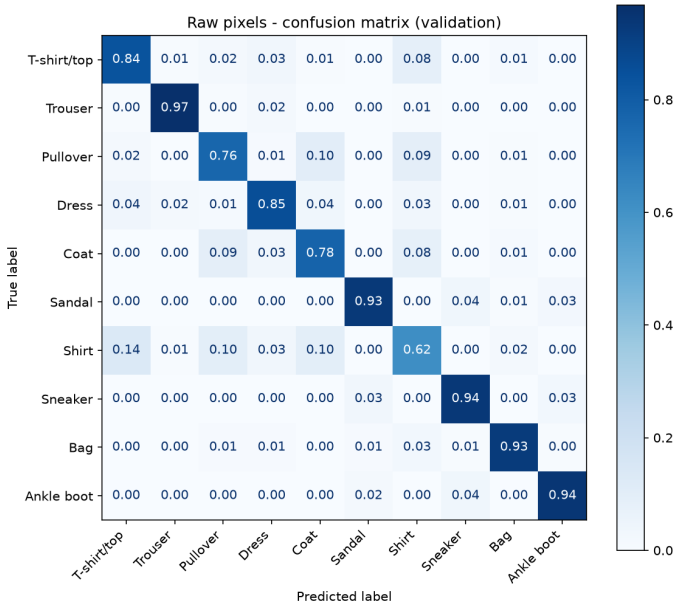


Figura 23: Matriz de confusão do classificador em píxeis brutos (validação).

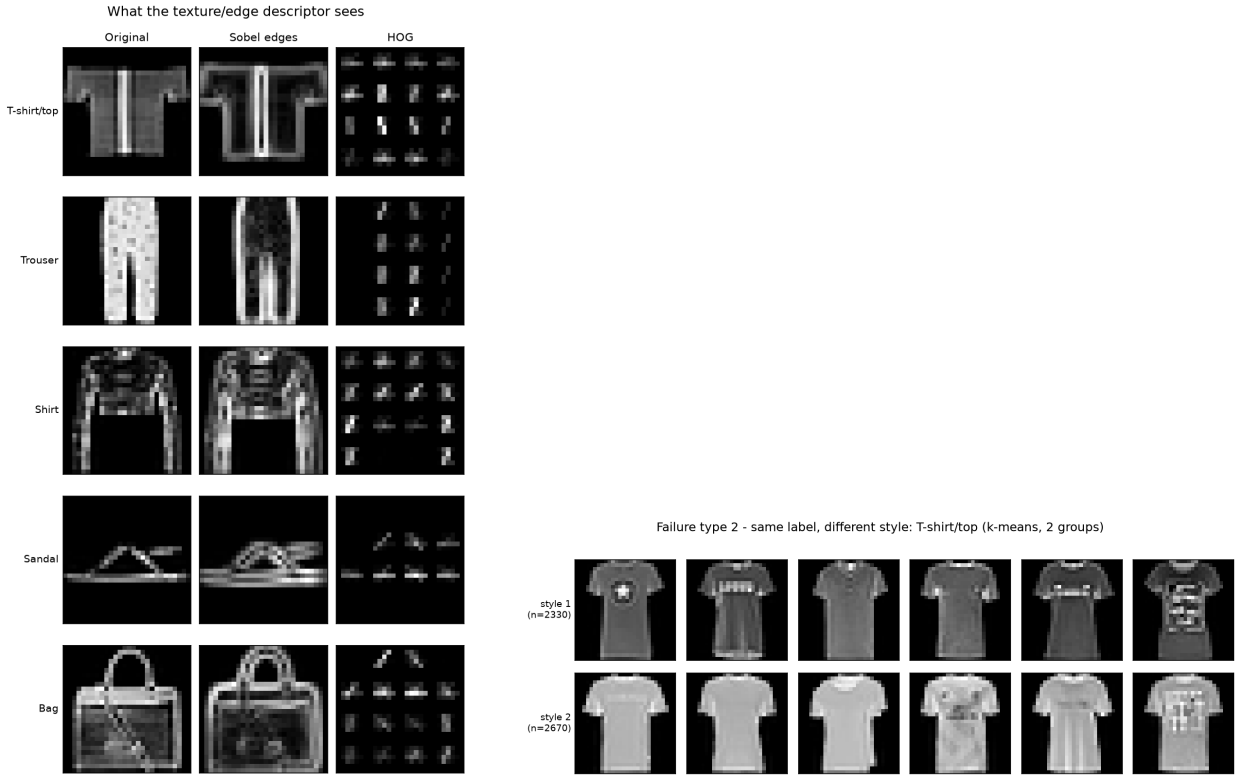


Figura 24: Esquerda: o que o Sobel e o HOG "veem". Direita: T-shirts separadas pelo k-means em escuras e claras (o brilho domina).